Universidade Positivo

DESENVOLVIMENTO FRONT-END PARA WEB

Aula: DFEW.11 – Persistência de Dados com LocalStorage.

Professor: Fabricio Olivo
fabricio@up.edu.br

PERSISTÊNCIA DE DADOS WEB

- Por padrão, o **JavaScript** em uma **página web** funciona em **memória volátil**.
- Isso significa que se o usuário **recarregar a página (F5)**, **fechar a aba** ou **navegar para outro site**, todas as **variáveis**, ***arrays*** e **objetos** criados pelo script **serão perdidos**.

A **Persistência de Dados** se faz pelo **Armazenamento Web Local** (*Client-Side Storage*) e permite **registrar informações** de forma **permanente** diretamente no **navegador** do usuário.

PERSISTÊNCIA DE DADOS WEB

- A persistência de dados no navegador é feita principalmente através de **localStorage**, **sessionStorage** e **cookies**, onde cada um possui **características**, **tempo de vida** e **casos de uso** específicos.

LOCALSTORAGE

- Tempo de vida: **Persistente**.
- Os **dados continuam salvos** mesmo após fechar e reabrir o navegador, só sendo apagados de forma manual (via código ou limpeza do browser).
- Capacidade: Cerca de 5 MB a 10 MB (varia por navegador).
- Comunicação com o servidor: Apenas no lado do cliente (client-side).
- Uso Web: Salvar **preferências do usuário** (como tema escuro/claro), **carrinhos de compras** ou **dados de usuário** logado e que precisam estar **disponíveis a longo prazo**.

PERSISTÊNCIA DE DADOS WEB

SESSIONSTORAGE

- Tempo de vida: **Temporário**.
- Os dados ficam disponíveis apenas enquanto a aba ou a janela do navegador estiver aberta; ao fechar, tudo é apagado.
- Capacidade: Cerca de 5 MB a 10 MB.
- Comunicação com o servidor: Apenas no lado do cliente (client-side).
- Uso Web: Armazenar **dados temporários de um formulário** em etapas ou informações que só fazem sentido **durante aquela sessão** específica.

PERSISTÊNCIA DE DADOS WEB

COOKIES

- Tempo de vida: Definido por uma **data de expiração** (Expires/Max-Age) ou válidos apenas **até o fim da sessão**.
- Capacidade: Pequena, em torno de 4 KB por cookie.
- Comunicação com o servidor: Enviados automaticamente pelo navegador em todas as requisições HTTP feitas para o domínio, o que pode impactar a performance se forem muito grandes.
- Uso Web: **Autenticação**, **controle de sessão** no servidor e **rastreamento** para recomendações de produtos personalizadas em *e-commerces*.

PERSISTÊNCIA DE DADOS WEB

COMANDOS BÁSICOS

```
// Salvar dados
localStorage.setItem('nome', 'Fabricio’); // chave, valor
sessionStorage.setItem('sessaoId', '123456’); // chave, valor

// Recuperar dados para serem manipulados por JS
const usuario = localStorage.getItem('nome’); // chave
const sessao = localStorage.getItem('sessaoId’); // chave

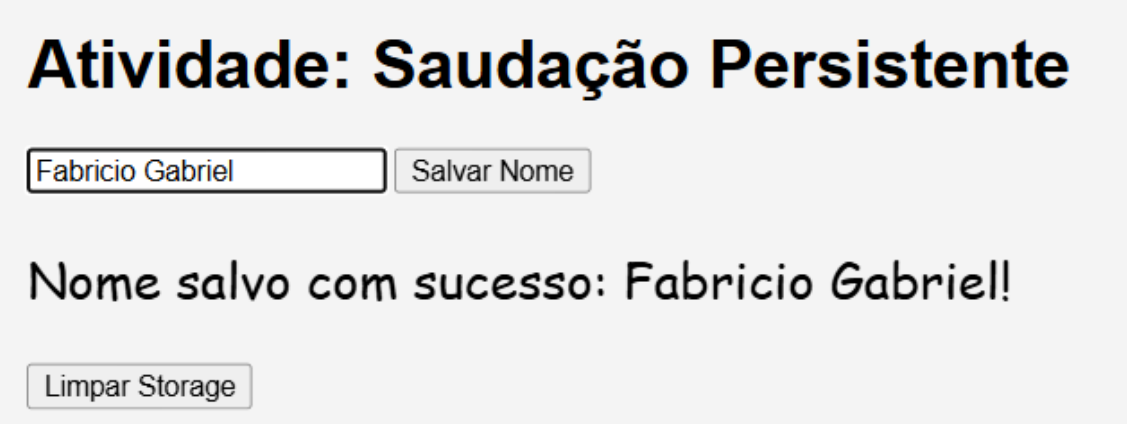
// Remover um item especifico
localStorage.removeItem('nome’); // chave
localStorage.removeItem('sessaoId’); // chave

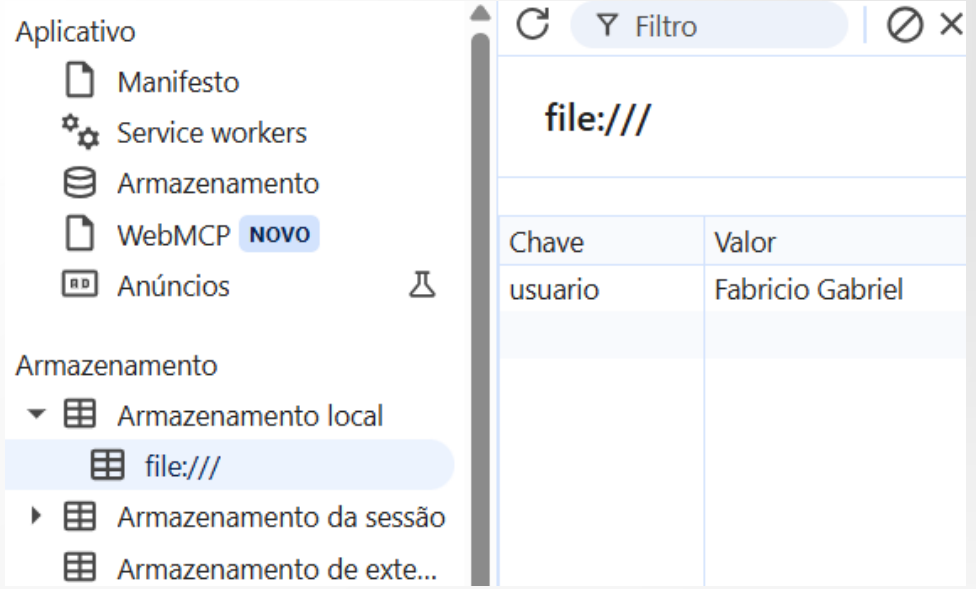
// Limpar todo o storage
localStorage.clear();
```

ATIVIDADE DA AULA

ATIVIDADE 1: SAUDAÇÃO PERSISTENTE

- Escreva um script em JS que ao clicar no botão “Salvar Nome” capture o nome de usuário do *input*, salve no *localStorage,* e exiba o nome capturado na página web com uma saudação, mesmo após atualizar ou fechar e abrir a janela, e que ao clicar no botão “Limpar Storage”, limpe todos os dados do *localStorage* e limpe a saudação, se existir.





ATIVIDADE DA AULA

```
const btn_s = document.getElementById('btn_Salvar');
const btn_l = document.getElementById('btn_Limpar');
const msn_s = document.getElementById('saudacao');
const nome = document.getElementById('nome');

const nome_salvo = localStorage.getItem('usuario');
msn_s.textContent = `Bem-vindo de volta, ${nome_salvo}!!!`;

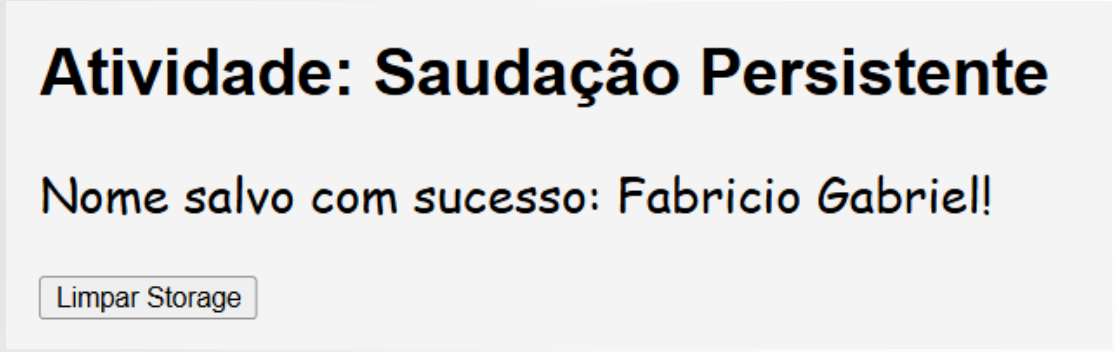
btn_s.addEventListener('click', salvarDados);
function salvarDados(){
    if (nome.value != "") {
        localStorage.setItem('usuario', nome.value);
        msn_s.textContent = `Nome salvo com sucesso: ${nome.value}!`;
        nome.value = "";
    } else { alert("Por favor, digite um nome válido."); }
}

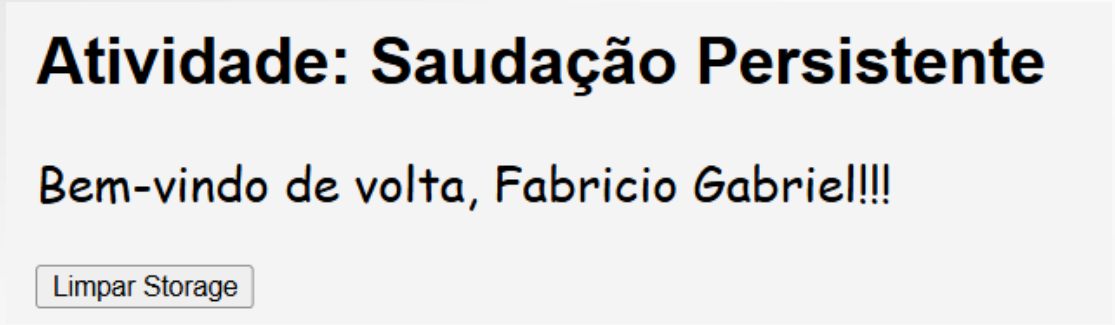
btn_l.addEventListener('click', limpaTudo);
function limpaTudo(){
    localStorage.clear();
    msn_s.textContent = "";
}
```

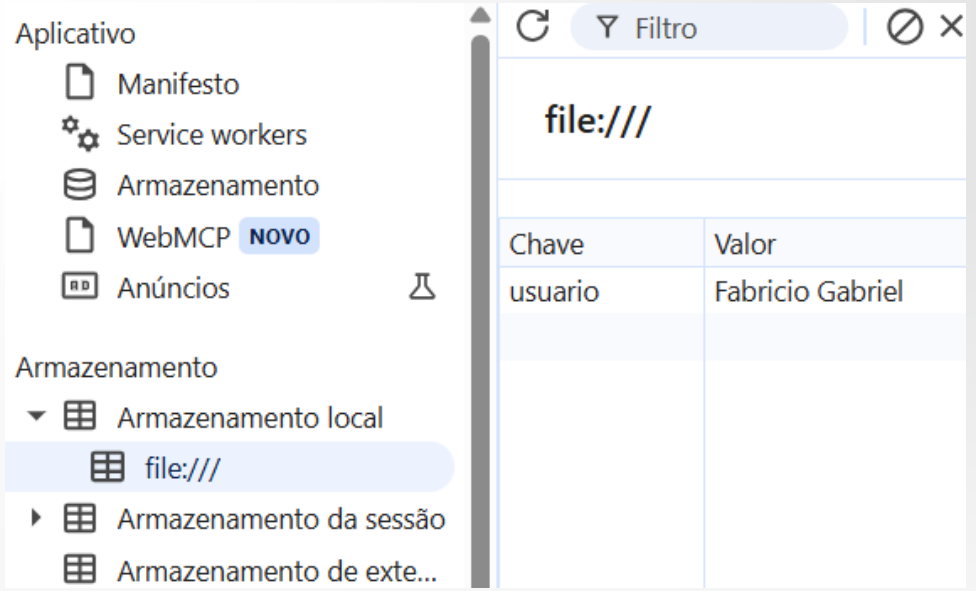
ATIVIDADE DA AULA

ATIVIDADE 2: SAUDAÇÃO PERSISTENTE

- Ajuste o script JS anterior para que quando existir um nome de usuário salvo no *localStorage*, exiba esse nome na página web com uma saudação e oculte o campo de input e o botão de "Salvar Nome".



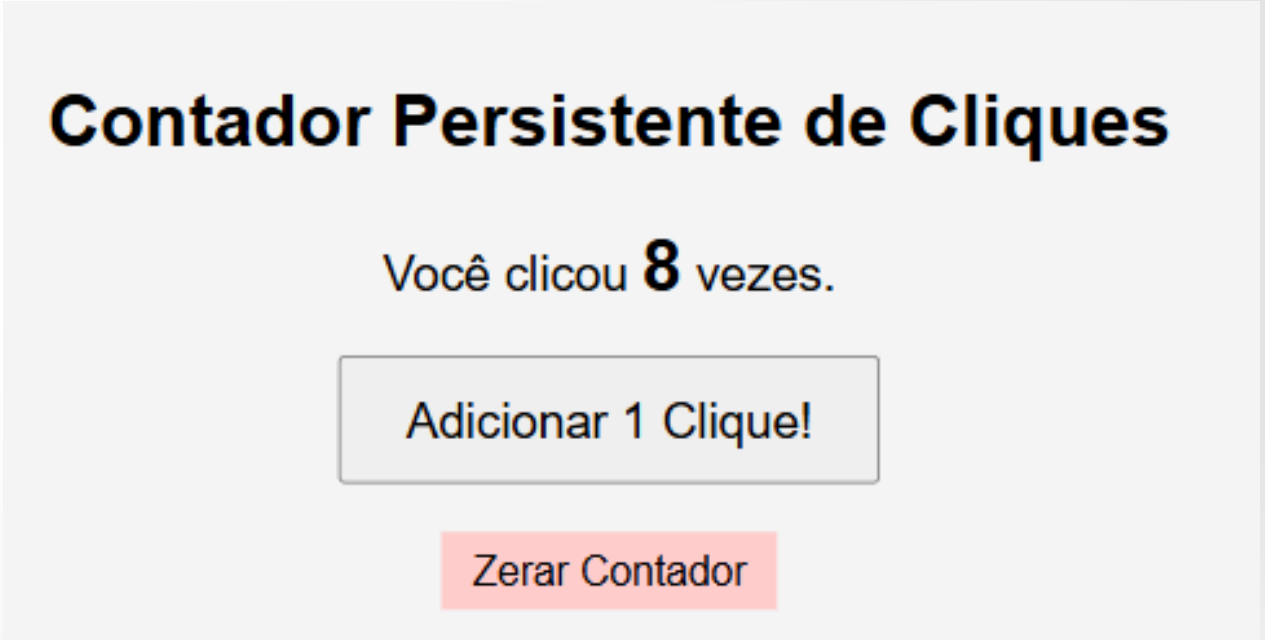




DESAFIO DA AULA

DESAFIO 1: CONTADOR PERSISTENTE DE CLIQUES

- Escreva um script em JS que ao clicar no botão “Adicionar 1 Clique" o número de vezes que o botão foi clicado apareça na tela de forma persistente, ou seja, mesmo depois da janela ter sido atualizada ou fechada e aberta novamente, devendo começar em zero.
- Use o recurso do **localStorage** para armazenar os dados de forma persistente.
- E ao clicar no botão "Zerar Contador" deve reiniciar a contagem a partir do zero.



PERSISTÊNCIA DE DADOS WEB

ARMAZENAR OBJETOS OU ARRAYS NO CLIENT-SIDE (NAVEGADOR)

- O ***storage*** só armazena ***strings*** (textos).
- Para **salvar dados** com **estruturas complexas** como **objetos** ou ***arrays***, você deverá **convertê-las para texto**, armazenar, recuperar, e **convertê-las para objeto JS** novamente para poderem ser **manipulados**.
- O **JavaScript** possui um **objeto global nativo** chamado **JSON** com dois métodos principais para **conversão** do formato de dados, para torná-los **compatíveis** e **manipuláveis**:
  - **JSON.parse()**: Converte uma **string JSON em um objeto JavaScript** real para você manipular.
  - **JSON.stringify()**: Converte um **objeto JavaScript em uma string JSON** para envio a uma **API**.
- O **JSON** (*JavaScript Object Notation*) é um formato leve de **texto** baseado na **sintaxe de objetos do JavaScript**, usado para **armazenar** e **trocar dados** entre sistemas e **APIs** (*Application Programming Interface*).

PERSISTÊNCIA DE DADOS WEB

ARMAZENAR OBJETOS OU ARRAYS NO CLIENT-SIDE (NAVEGADOR)

```
// Cria dois objetos JS
let userJS = { nome: 'Gabriel', idade: 30, altura: 1.85 };
let listaJS = ["Fabricio", "Gabriel", "Olivo", 30, 1.85];

// Converte objetos JS para String (Texto) JSON
let userText = JSON.stringify(userJS);
let listaText = JSON.stringify(listaJS);

// Armazena os textos JSON no storage do navegador
localStorage.setItem('user', userText);
localStorage.setItem('lista', listaText);

// Recupera os textos JSON e converte para objetos JS manipuláveis
let userObjJS = JSON.parse(localStorage.getItem('user'));
let listaObjJS = JSON.parse(localStorage.getItem('lista'));
```

ATIVIDADE DA SEMANA

ATIVIDADE 3: TEMA (DARK/LIGHT MODE) PERSISTENTE

- Escreva um **script** em **JS** que ao clicar no botão “Alternar Modo” alterne o modo de tela entre **Dark** e **Light** com **persistência**, ou seja, que se mantenha como preferência do usuário na atualização ou fechamento e abertura da janela.
- Use o recurso do **localStorage** para armazenar os dados de forma persistente.

DESAFIO DA SEMANA

DESAFIO 2: **BLOCO DE NOTAS PERSISTENTE**

- Escreva um **script** em **JS** que o usuário digita um texto em um *textarea*, clica no botão "Salvar Anotação", e a nota é adicionada a um ***array*** persistente.
- Use o recurso do **localStorage** para armazenar os dados de forma persistente.
- As **notas** salvas devem aparecer em **formato** de **Tabela** logo abaixo dos outros elementos, onde na **primeira coluna** terá a nota armazenada e na **segunda coluna** terá um botão para excluir a referida nota.
- Use os comandos **JSON.stringify()** e **JSON.parse()** para permitir gravação e leitura de *Array* (lista) no **localStorage.**

TUTORIAIS E GUIAS

- Confira essas matérias sobre os fundamentos da WEB:
  - Um guia para iniciantes na área de desenvolvimento web.
  - W3C: Consórcio que regulamenta os padrões para web.

- Por aqui você começará a entender como são estruturadas as interfaces das páginas dos sites na Internet:
  - https://developer.mozilla.org/pt-BR/docs/Learn/HTML/Introduction_to_HTML/Getting_started
  - https://www.w3schools.com/
  - Projeto Khan Academy para Programação

- Faça seus testes no desenvolvimento WEB usando ambientes online preparados pra isso:
  - jsbin.com – Testes de Front-End (html, css e js);

BIBLIOGRAFIA BÁSICA

- FLATSCHART, F. **HTML 5: embarque imediato**. Rio de Janeiro: Brasport, 2011. E-book.
- SANDERS, B. **Smashing HTML5: técnicas para a nova geração da web**. Porto Alegre: Bookman, 2012. E-book.
- TERUEL, E. C. **HTML5: guia prático**. 2. ed. São Paulo: Érica, 2014. E-book.

BIBLIOGRAFIA COMPLEMENTAR

- BERTAGNOLLI, S.; MILETTO, E. M. (org.). **Desenvolvimento de software II: introdução ao desenvolvimento web com HTML, CSS, JavaScript e PHP**. Porto Alegre: Bookman, 2014. E-book.
- HAROLD, E. R. **Refatorando HTML: como melhorar o projeto de aplicações web existentes**. Porto Alegre: Bookman, 2010. E-book.
- HORSTMANN, C. **Conceitos de computação com Java**. 5. ed. Porto Alegre: Bookman, 2009. E-book.
- KALBACH, J. **Design de navegação web: otimizando a experiência do usuário**. Porto Alegre: Bookman, 2009. E-book.
- MACHADO, R. P.; FRANCO, M. H. I.; BERTAGNOLLI, S. C. **Desenvolvimento de software III: programação de sistemas web orientada a objetos em Java**. Porto Alegre: Bookman, 2016. E-book.